TP (Partie1) : Mise en œuvre d'un pipeline ETL et d'un Data Lakehouse (Bronze/Silver/Gold)

1. Contexte et objectifs

E-shop est une entreprise de commerce qui souhaite mettre en place une chaîne de traitement de données permettant de contrôler la qualité de ses sources, de consolider les informations commerciales et de produire des indicateurs fiables.

Le jeu de données de l'E-shop est composé de cinq sources : les clients (`customers`), qui contiennent les informations relatives aux clients ; les produits (`products`), qui représentent le catalogue des produits commercialisés ; les commandes (`orders`), qui enregistrent les achats effectués par les clients ; les lignes de commande (`order_items`), qui détaillent les produits achetés, leurs quantités et leurs prix unitaires pour chaque commande ; et les promotions (`promotions`), qui regroupent les codes promotionnels ainsi que leurs taux de réduction et leurs périodes de validité.

Ces fichiers contiennent volontairement des anomalies. Votre mission consiste à concevoir et à mettre en œuvre un pipeline de Data Engineering avec Apache Spark et PySpark, depuis l'ingestion des données jusqu'à la production des indicateurs analytiques.

À la fin du TP, vous devrez être capables de :

- ingérer des fichiers CSV et JSON Lines avec Spark ;
- définir et contrôler les schémas des sources ;
- détecter les anomalies, dédupliquer les données et isoler les enregistrements invalides ;
- construire un Data Lakehouse structuré selon les zones Bronze, Silver et Gold ;
- effectuer des jointures et des transformations distribuées ;
- exploiter Spark SQL et les fonctions de fenêtrage ;
- produire des indicateurs commerciaux et vérifier leur cohérence ;
- assurer la persistance, la reproductibilité et l'idempotence du pipeline ;

Consigne générale : Réalisez le TP dans un notebook Jupyter avec PySpark, en organisant clairement les différentes étapes du pipeline ETL et de l'architecture de données Bronze/Silver/Gold. Utilisez les fichiers sources fournis en annexe tels quels, sans corriger ni supprimer manuellement les anomalies avant leur ingestion. Toutes les opérations de nettoyage, de validation, de correction et de gestion des anomalies doivent être intégrées au pipeline, documentées et traçables.

2. Jeu de données et règles métier

2.1. Sources fournies

Fichier	Format	Contenu
customers.csv	CSV	Identifiants, noms, régions et dates d'inscription
products.csv	CSV	Référentiel des produits, catégories, prix et coûts
orders.csv	CSV	Commandes, clients, dates, statuts et codes promotionnels
order_items.csv	CSV	Lignes de commande, produits, quantités et prix unitaires
promotions.jsonl	JSON Lines	Codes promotionnels, taux de remise et périodes de validité

Les quatre fichiers CSV utilisent le point-virgule (`;`) comme séparateur. Le fichier `promotions.jsonl` contient un objet JSON par ligne.

2.2. Règles métier

Les règles suivantes doivent être appliquées dans l'ensemble du pipeline.

Règle 1 : Éligibilité des commandes

Une commande est éligible au calcul des ventes lorsque son statut vaut `PAID` ou `DELIVERED`.

Les commandes `CANCELLED` et `PENDING` sont exclues des indicateurs de ventes. Cette exclusion relève d'une règle métier et ne signifie pas, à elle seule, que les enregistrements sont de mauvaise qualité.

Les commandes `PENDING` et `CANCELLED` dont les données sont par ailleurs valides doivent être conservées dans la couche Silver. Elles ne doivent pas être placées en quarantaine en raison de leur seul statut. Leur exclusion des indicateurs commerciaux est appliquée lors de la construction de la couche Gold.

Règle 2 : Intégrité référentielle

- Chaque commande doit référencer un client valide.
- Chaque ligne de commande doit référencer une commande et un produit valides.
- Une référence inexistante doit être signalée et traitée selon une règle explicite de rejet ou de quarantaine.
- Une ligne rejetée ne doit pas contribuer aux indicateurs Gold.

Règle 3 : Gestion des promotions

Une remise est applicable uniquement si le code promotionnel existe dans le référentiel et si la date de commande appartient à la période de validité de la promotion.

Appliquez les règles suivantes :

- **Aucune promotion renseignée :** taux de remise égal à zéro.
- **Promotion connue et valide à la date de commande :** appliquer le taux de remise défini dans le référentiel.
- **Promotion connue mais expirée ou pas encore valide :** taux de remise égal à zéro et anomalie métier documentée.
- **Code promotionnel inconnu :** taux de remise égal à zéro et anomalie documentée ; la commande n'est pas automatiquement exclue des ventes pour cette seule raison.

Une commande peut donc être éligible aux ventes tout en comportant une anomalie relative à sa promotion.

Règle 4 : Prix transactionnel et prix catalogue

Le champ `order_items.unit_price` correspond au prix unitaire enregistré au moment de la transaction.

Le champ `products.unit_price` correspond au prix unitaire du catalogue.

Vous devez expliquer la différence entre ces deux valeurs. Le prix transactionnel doit être utilisé pour calculer le chiffre d'affaires ; il ne doit pas être remplacé silencieusement par le prix catalogue. Tout écart significatif entre les deux doit pouvoir être analysé.

Règle 5 : Calculs financiers

Le chiffre d'affaires net est calculé à partir du prix transactionnel, de la quantité et de la remise applicable :

CA net = somme de quantité × prix unitaire transactionnel × (1 − taux de remise applicable)

Le coût total est calculé à partir du coût unitaire du produit :

Coût total = somme de quantité × coût unitaire du produit

La marge est définie par :

Marge = CA net − Coût total

Le taux de marge sur chiffre d'affaires est :

Taux de marge = Marge totale / CA net total × 100

Le panier moyen est :

Panier moyen = CA net total / nombre de commandes éligibles

Précisez le comportement attendu si un dénominateur est nul. Utilisez des types numériques adaptés aux calculs financiers et documentez la règle d'arrondi retenue.

Règle 6 : Qualité, déduplication et règles métier

Une anomalie de qualité ne doit pas être confondue avec une exclusion métier.

La déduplication doit être explicite, reproductible et traçable :

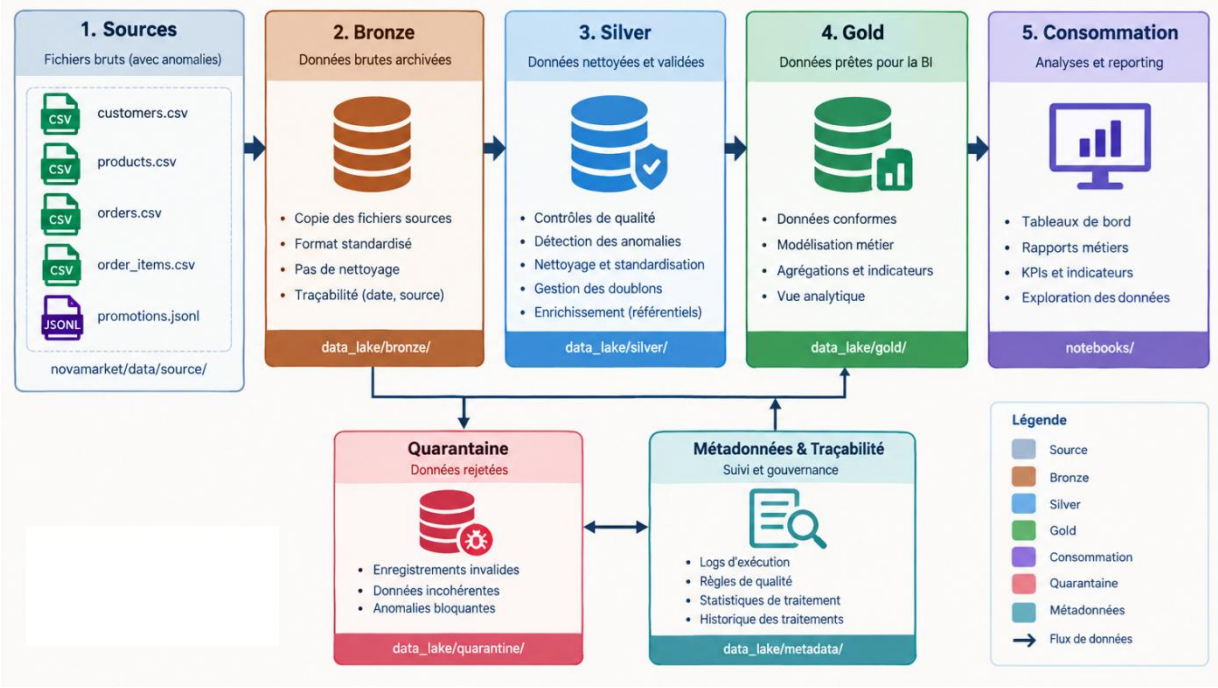
- **Doublons strictement identiques :** conserver une seule occurrence canonique dans la table Silver, identifier le nombre de copies supplémentaires et enregistrer dans un fichier d'audit le contenu de la ligne, sa clé d'identification et le nombre de copies supplémentaires.
- **Enregistrements ayant le même identifiant mais des contenus différents :** appliquer une règle de résolution justifiée ou placer les enregistrements en conflit en quarantaine.
- **Conservation des données originales :** les fichiers sources doivent rester intégralement conservés dans Bronze. Il n'est pas nécessaire de distinguer individuellement les occurrences physiquement identiques ; leur suivi peut être réalisé par groupe de lignes identiques, à condition de garantir la traçabilité des copies supplémentaires et la conservation des volumes.
- **Éviter les suppressions arbitraires :** ne pas supprimer toutes les occurrences d'un identifiant dupliqué si une occurrence valide peut être conservée.

Cette règle est importante pour préserver l'intégrité des références entre clients, commandes, produits et lignes de commande.

Toute hypothèse complémentaire doit être documentée et appliquée de façon cohérente.

3. Architecture cible

Le pipeline doit respecter l’organisation logique suivante :



Bronze : Conservation

- Conservation des données reçues.
- Traçabilité des sources.
- Manifeste d’ingestion.
- Possibilité de rejouer les traitements.

Silver : Qualité et validation

- Conversion des types.
- Contrôle des champs obligatoires.
- Détection des doublons.
- Validation des dates et des valeurs numériques.
- Contrôle de l’intégrité référentielle.
- Conservation des motifs de rejet.

Gold : Exploitation analytique

- Ventes éligibles.
- Chiffre d’affaires, coûts et marges.
- Agrégations par catégorie et par période.

- Classements de produits et analyses clients.

Metadata : Traçabilité

Conservation des informations d'ingestion, des volumes traités, des contrôles réalisés et des éléments permettant de reproduire les traitements.

La quarantaine est une zone distincte. Elle ne remplace ni Bronze ni Silver. Les données brutes doivent rester disponibles, même lorsqu'elles contiennent des anomalies.

4. Étape 1 : Préparation de l'environnement et ingestion

Travail demandé

1. Créez un projet nommé `E-shop` et organisez son arborescence conformément à la structure proposée dans l'annexe. Celle-ci doit prévoir les dossiers nécessaires aux sources de données, aux scripts de traitement, aux données persistées et aux livrables.
2. Initialisez une session Spark.
3. Affichez la version de Spark, le master utilisé et l'identifiant de l'application.
4. Lisez les cinq sources dans des DataFrames.
5. Définissez les schémas attendus et justifiez les types retenus pour les identifiants, les dates, les quantités, les prix et les taux de remise.

   Précision sur les schémas : distinguez les types des données lors de leur lecture depuis les sources des types métier attendus après conversion et validation. Pour les fichiers CSV, privilégiez une lecture initiale des champs sous forme de chaînes de caractères (`StringType`), afin de préserver les valeurs originales et de faciliter la détection des erreurs de conversion. Les conversions vers les types métier attendus seront réalisées explicitement lors de la préparation des données dans l'étape 3 (Silver).

6. Affichez les schémas et quelques lignes de chaque source.
7. Comptez les enregistrements de chaque fichier et comparez les volumes aux indications de l'annexe.
8. Identifiez les difficultés liées aux valeurs nulles, aux champs vides, aux conversions numériques et aux dates invalides.

Questions d'analyse

- Pourquoi est-il utile de contrôler les schémas avant les transformations ?
- Quelle différence existe-t-il entre une lecture réussie et une ingestion de qualité ?
- Pourquoi les données brutes doivent-elles rester disponibles ?
- Pourquoi peut-il être préférable de lire initialement les champs CSV comme des chaînes avant de convertir et de contrôler leurs types ?

Livrable : configuration de l'environnement, schémas retenus, code d'ingestion et rapport initial sur les volumes.

5. Étape 2 : Zone Bronze

Travail demandé

1. Conservez les cinq fichiers sources dans la zone Bronze en préservant leur contenu d'origine, sans modification ni suppression des anomalies.
2. Vous pouvez également produire une représentation des données au format Parquet, afin de faciliter leur exploitation avec Spark. Cette représentation ne doit pas remplacer les fichiers sources originaux.
3. Organisez et documentez les chemins de stockage des fichiers originaux et des éventuelles représentations Parquet.
4. Créez un manifeste d'ingestion pour chaque source de données, contenant au minimum les informations suivantes :

    - le nom de la source ;
    - le nombre de lignes ingérées ;
    - la date et l'heure de l'ingestion ;
    - le chemin de stockage des données dans la couche Bronze ;
    - le statut de l'ingestion (succès ou échec) ;
    - un identifiant unique de l'exécution (`run_id`).

5. Enregistrez le manifeste dans un format structuré (JSON ou CSV), dans un emplacement dédié du Data Lakehouse.

N.B : `run_id` permet de rattacher les données ingérées à une exécution précise du pipeline et facilite la traçabilité et l'audit des traitements.

6. Vérifiez que les données écrites peuvent être relues et que les volumes sont cohérents.

Questions

- Pourquoi Bronze ne doit-elle pas supprimer les anomalies détectées ?
- Comment cette zone facilite-t-elle le rejeu d'un pipeline ?
- Quelles informations permettent de retracer une ingestion ?
- Quelle différence existe entre conserver un fichier source et le convertir en Parquet ?

Livrable : données Bronze, manifeste d'ingestion et contrôles de relecture.

6. Étape 3 : Contrôle de la qualité et gestion des anomalies

6.1. Objectifs

Vous devez identifier les anomalies, définir des règles de qualité adaptées à chaque source, séparer les données valides des données rejetées et vérifier que le traitement ne provoque aucune perte inexpliquée de lignes.

Source des données : les traitements de cette étape doivent être réalisés à partir des données préalablement stockées dans la zone Bronze. Il est interdit de relire directement les fichiers sources initiaux pour construire la zone Silver. Les données validées et nettoyées seront enregistrées dans Silver, tandis que les enregistrements rejetés seront conservés séparément dans une zone de quarantaine, avec leurs motifs de rejet.

Pour chaque source, distinguez :

- les lignes valides ;
- les lignes invalides mises en quarantaine ;
- les copies supplémentaires résultant de doublons exacts ;
- les anomalies métier qui ne rendent pas nécessairement la ligne structurellement invalide.

Pour des doublons strictement identiques, une seule occurrence canonique est conservée et les copies supplémentaires sont tracées dans une sortie d'audit. Pour des enregistrements ayant le même identifiant mais des contenus différents, les occurrences en conflit doivent faire l'objet d'une règle de résolution justifiée ou être placées en quarantaine. Une même ligne ne doit jamais être comptabilisée dans plusieurs catégories de sortie.

6.2. Exemple guidé — Contrôler `customers.csv`

Le fichier `customers.csv` contient les informations relatives aux clients.

Champ	Contrôle à réaliser
customer_id	Obligatoire et unique dans la table canonique
customer_name	Obligatoire et non vide
region	Obligatoire et non vide
registration_date	Doit être convertible en date

Question 1 : Charger la source

Chargez le fichier avec le séparateur `;` et conservez initialement les données sous forme de chaînes.

```
from pyspark.sql import functions as F

customers_raw = (
    spark.read
    .option("header", True)
    .option("sep", ";")
    .option("inferSchema", False)
    .csv("data_lakehouse/bronze/customers.csv")
)

customers_raw.printSchema()
customers_raw.show(10, truncate=False)

print("Nombre de lignes :", customers_raw.count())
```

Interprétez les résultats et vérifiez les noms de colonnes obtenus.

Question 2 : Vérifier les champs obligatoires

Recherchez les valeurs nulles ou vides dans `customer_id`, `customer_name` et `region`.

```
customers_raw.select(
    F.sum(
        F.when(
            F.col("customer_id").isNull()
            | (F.trim(F.col("customer_id")) == ""),
            1
```

```
        ).otherwise(0)
    ).alias("id_manquant"),

    F.sum(
        F.when(
            F.col("customer_name").isNull()
            | (F.trim(F.col("customer_name")) == ""),
            1
        ).otherwise(0)
    ).alias("nom_manquant"),

    F.sum(
        F.when(
            F.col("region").isNull()
            | (F.trim(F.col("region")) == ""),
            1
        ).otherwise(0)
    ).alias("region_manquante")
).show()
```

Expliquez ce que mesure chaque indicateur.

Question 3 : Contrôler les dates

Convertissez `registration_date` en type date tout en conservant la valeur d'origine.

```
customers_typed = customers_raw.withColumn(
    "registration_date_parsed",
    F.to_date("registration_date", "yyyy-MM-dd")
)

customers_typed.select(
    "customer_id",
    "registration_date",
    "registration_date_parsed"
).show(truncate=False)
```

Identifiez les lignes dont la date est renseignée mais dont la conversion échoue. Vérifiez le comportement de votre version de Spark pour les dates invalides et documentez la configuration utilisée.

Question 4 : Détecter les doublons

Repérez les identifiants apparaissant plusieurs fois.

```
duplicate_ids = (
    customers_raw
    .groupBy("customer_id")
    .count()
    .filter(F.col("count") > 1)
)

duplicate_ids.show(truncate=False)
```

Examinez les lignes correspondantes et distinguez les doublons strictement identiques des conflits entre enregistrements.

Consigne : ne placez pas automatiquement toutes les occurrences d'un identifiant dupliqué en quarantaine. Si deux lignes sont strictement identiques, une seule peut être conservée comme enregistrement canonique ; la copie supplémentaire doit être tracée. Si les lignes diffèrent, justifiez la règle appliquée.

Question 5 : Construire une justification des anomalies

Créez une colonne `quality_reason` indiquant les anomalies détectées. Plusieurs motifs peuvent être associés à une même ligne.

Le squelette suivant illustre la construction des motifs liés aux champs obligatoires et aux dates. Complétez-le avec la gestion des doublons selon la règle précédente.

```
customers_checked = customers_typed.withColumn(
    "quality_reason",
    F.concat_ws(
        " | ",
        F.when(
            F.col("customer_id").isNull()
            | (F.trim(F.col("customer_id")) == ""),
            F.lit("ID_MANQUANT")
        ),
        F.when(
            F.col("customer_name").isNull()
            | (F.trim(F.col("customer_name")) == ""),
            F.lit("NOM_MANQUANT")
        ),
        F.when(
            F.col("region").isNull()
            | (F.trim(F.col("region")) == ""),
            F.lit("REGION_MANQUANTE")
        ),
        F.when(
            F.col("registration_date").isNotNull()
            & (F.trim(F.col("registration_date")) != "")
            & F.col("registration_date_parsed").isNull(),
            F.lit("DATE_INVALIDE")
        )
    )
)
```

Question 6 : Séparer les données et vérifier leur conservation

Construisez les DataFrames suivants :

- `customers_valid` : enregistrements canoniques respectant les règles de qualité ;
- `customers_quarantine` : enregistrements invalides ;
- une sortie d'audit des doublons exacts écartés, si nécessaire.

Vérification de la conservation des lignes

Vérifiez que chaque ligne initialement ingérée est affectée à une seule et unique catégorie de sortie : enregistrement canonique valide, enregistrement en quarantaine ou copie supplémentaire de doublon exact tracée dans l'audit.

Vérifiez que toutes les lignes initialement ingérées sont prises en compte dans les différentes sorties du traitement, sans perte ni double comptabilisation.

Appliquez la règle de conservation suivante :

Nombre initial de lignes = nombre d'enregistrements canoniques valides + nombre d'enregistrements en quarantaine + nombre de copies supplémentaires de doublons exacts tracées.

Précisions :

- Un enregistrement canonique valide est conservé dans `customers_valid`.
- Un enregistrement invalide est placé dans `customers_quarantine`.
- Une copie supplémentaire résultant d'un doublon strictement identique est enregistrée dans une sortie d'audit dédiée. Elle n'est comptabilisée ni dans les enregistrements canoniques ni dans la quarantaine.
- En cas de conflit entre des enregistrements portant le même identifiant, les lignes concernées sont traitées selon la règle de résolution justifiée ou placées en quarantaine.

Les trois catégories utilisées dans l'équation doivent être mutuellement exclusives : une même ligne ne doit être comptabilisée que dans une seule catégorie.

Vérifiez cette égalité à l'aide des comptages obtenus avec Spark et expliquez tout écart éventuel.

Question 7 : Préparer Silver

Enregistrez les données validées dans Silver et les lignes invalides dans la zone de quarantaine. Conservez les motifs de rejet, les valeurs d’origine et les informations nécessaires à l’audit.

6.3. Travail autonome : Appliquer la méthode aux autres sources

A. products.csv : Référentiel des produits

Contrôles minimaux :

- identifiant produit obligatoire ;
- nom et catégorie renseignés ;
- prix catalogue et coût unitaire numériques et non négatifs ;
- détection des identifiants dupliqués ;
- détection des conflits entre enregistrements portant le même identifiant.

Questions :

1. Quelles lignes doivent être mises en quarantaine ?
2. Une valeur numérique égale à zéro est-elle toujours invalide ?
3. Pourquoi un coût négatif pose-t-il un problème pour les calculs de marge ?

B. orders.csv : Commandes

Contrôles minimaux :

- identifiant de commande obligatoire et unique ;
- identifiant client renseigné et existant dans Silver ;
- date de commande valide ;
- statut renseigné et appartenant aux statuts reconnus dans le sujet ;
- code promotionnel éventuellement absent ;
- code promotionnel inconnu signalé distinctement d'un code connu mais expiré.

Questions :

1. Une commande `PENDING` est-elle nécessairement invalide ?
2. Comment distinguer une commande structurellement invalide d'une commande exclue des ventes ?
3. Quel traitement appliquer à une commande référençant un client inexistant ?

C. order_items.csv : Lignes de commande

Contrôles minimaux :

- identifiant de ligne obligatoire et unique ;
- commande référencée existante ;
- produit référencé existant ;
- quantité entière strictement positive ;
- prix de vente unitaire numérique et non négatif.

Questions :

1. Comment détecter les références inexistantes ?
2. Pourquoi ne faut-il pas remplacer silencieusement une quantité invalide par zéro ?
3. Comment éviter qu'un doublon de ligne augmente artificiellement le chiffre d'affaires ?

D. Promotions (`promotions.jsonl`)

Contrôles minimaux :

Le traitement du fichier `promotions.jsonl` doit couvrir les contrôles suivants :

- **Validité syntaxique JSON :** chaque ligne doit pouvoir être analysée comme un objet JSON valide. Les lignes mal formées doivent être détectées et placées en quarantaine, en conservant leur contenu brut et le motif du rejet.
- **Conversion et typage :** les valeurs des champs doivent être convertibles vers les types attendus (par exemple, un taux de remise non numérique ou une date non convertible).
- **Validation métier :** vérifier la présence et l'unicité du code promotionnel, la validité du taux de remise (intervalle [0, 1]) ainsi que la validité et la cohérence des dates de validité.

Méthode de lecture et de traitement des lignes JSONL

Le fichier `promotions.jsonl` contient un objet JSON par ligne et comporte volontairement une ligne mal formée. Son traitement doit permettre de détecter cette anomalie sans perdre les données originales.

Procédez selon les étapes suivantes :

1. **Lecture des données brutes :** lisez le fichier comme un fichier texte à l'aide de `spark.read.text()`. Chaque ligne doit être conservée sous forme de chaîne de caractères, sans correction ni suppression préalable.
2. **Analyse du contenu JSON :** utilisez les fonctions de parsing JSON de Spark, notamment `from_json()`, pour analyser individuellement chaque ligne selon le schéma attendu. Les lignes dont le contenu ne peut pas être interprété comme un objet JSON valide doivent être identifiées et placées en quarantaine.

   Précaution concernant le parsing JSON : Pour faciliter la distinction entre les erreurs de syntaxe JSON et les erreurs de conversion des champs, privilégiez un schéma de parsing initial permettant de préserver les valeurs des champs sous forme de chaînes. Effectuez ensuite les conversions explicites vers les types métier attendus.

3. **Conversion et validation :** pour les lignes correctement analysées, convertissez explicitement les champs vers les types attendus, puis appliquez les contrôles de conversion, de typage et de validation métier définis précédemment. Les différentes catégories d'erreurs doivent être distinguées et documentées.

Pour chaque ligne, conservez :

- le contenu brut original ;
- le résultat de l'analyse JSON ;
- un indicateur de validité du parsing ;
- les valeurs converties, le cas échéant ;
- le motif de rejet ou d'anomalie, le cas échéant.

Attention : aucune ligne ne doit être supprimée silencieusement. Les lignes mal formées ou présentant des erreurs de conversion ou de validation doivent être conservées dans les structures de quarantaine ou d'audit appropriées, avec leur contenu original et les motifs associés, afin de garantir la traçabilité du traitement.

Questions :

1. Comment détecter et traiter une ligne JSON mal formée ? Pourquoi faut-il conserver son contenu brut ?
2. Comment vérifier qu'une promotion est applicable à la date d'une commande ?
3. Quelle différence existe entre une promotion absente, inconnue et expirée ?

6.4. Tableau de synthèse qualité

Pour chaque source, produisez un tableau comprenant :

Indicateur	Résultat
Nombre de lignes initiales	À mesurer
Nombre d'enregistrements canoniques valides	À mesurer
Nombre de lignes invalides en quarantaine	À mesurer
Nombre de copies supplémentaires tracées	À mesurer
Nombre de conflits d'identifiants	À mesurer

Nombre de références inexistantes	À mesurer, si applicable
Principaux motifs de rejet	À documenter
Conservation des lignes vérifiée	Oui / Non

Les résultats doivent provenir de vos traitements Spark, et non d'un comptage manuel des anomalies de l'annexe.

6.5. Questions de synthèse

1. Pourquoi les règles de qualité diffèrent-elles d'une source à l'autre ?
2. Quelle différence existe entre une anomalie de qualité, une référence inexistante et une exclusion métier ?
3. Pourquoi conserver les données rejetées et leurs motifs ?
4. Comment vérifier que les traitements de contrôle sont reproductibles ?
5. Quelles règles doivent être validées avant de calculer le chiffre d'affaires ?

Livrable : code de contrôle des cinq sources, données Silver, quarantaine, sorties d'audit et rapport de qualité.

7. Étape 4 : Jointures et construction de la vue commerciale

Travail demandé

1. Reliez les lignes de commande aux commandes.
2. Reliez les lignes de commande aux produits pour récupérer la catégorie et le coût unitaire.
3. Reliez les commandes aux clients lorsque l'analyse l'exige.
4. Rattachez les promotions par `promotion_code`.
5. Vérifiez la validité temporelle des promotions à la date de commande.
6. Distinguez les commandes structurellement valides des commandes éligibles aux ventes.
7. Construisez une vue analytique détaillée permettant de calculer le chiffre d'affaires net, le coût et la marge.
8. Vérifiez le nombre de lignes avant et après chaque jointure.
9. Vérifiez l'unicité des clés dans les référentiels avant les jointures.
10. Documentez les éventuelles lignes exclues et leur motif.

Une jointure avec une clé dupliquée peut multiplier les lignes et gonfler artificiellement les KPI. La cardinalité des jointures doit donc être contrôlée avant les agrégations.

Questions d'analyse

- Quelle différence existe entre une jointure interne et une jointure externe ?
- Dans quelles conditions une jointure peut-elle augmenter le nombre de lignes ?
- Pourquoi une jointure avec un référentiel comportant des clés dupliquées peut-elle fausser les KPI ?
- Pourquoi contrôler la cardinalité avant l'agrégation ?
- Quelle différence existe entre une promotion inconnue, une promotion expirée et l'absence de promotion ?
- Comment vérifier que les prix transactionnels ont été conservés ?

Livrable : DataFrames de jointure, contrôles de cardinalité et description des règles de calcul.

8. Étape 5 : Zone Gold et indicateurs commerciaux

8.1. Construction de Gold

À partir des données Silver et des règles métier :

1. Construisez une table Gold détaillée au niveau de la ligne de commande.
2. Ne retenez que les commandes dont le statut est `PAID` ou `DELIVERED`. Cette sélection ne remet pas en cause la conservation dans Silver des commandes `PENDING` et `CANCELLED` dont les données sont valides. La couche Silver représente les données validées, tandis que la couche Gold applique les règles d'éligibilité aux indicateurs commerciaux.
3. Appliquez le taux de remise déterminé par les règles métier.
4. Calculez le chiffre d'affaires net, le coût et la marge de chaque ligne.
5. Conservez les identifiants nécessaires à la traçabilité.
6. Enregistrez les résultats dans des tables Gold persistées.

8.2. Indicateurs globaux

Calculez :

- le nombre de commandes éligibles ;
- le nombre de lignes Gold ;
- le nombre de clients concernés ;
- la quantité totale vendue ;
- le chiffre d'affaires net ;
- le coût total ;
- la marge totale ;
- le taux de marge sur le chiffre d'affaires ;
- le panier moyen par commande éligible.

Expliquez les formules et les dénominateurs utilisés. Le nombre de commandes doit être calculé sur les identifiants distincts, et non en comptant les lignes de commande.

8.3. Analyse par catégorie

Pour chaque catégorie, calculez :

- la quantité vendue ;
- le nombre de lignes de commande ;
- le chiffre d'affaires net ;
- le coût total ;
- la marge ;
- le taux de marge.

Identifiez les catégories contribuant le plus au chiffre d'affaires et à la marge. Expliquez pourquoi la catégorie générant le chiffre d'affaires le plus élevé n'est pas nécessairement celle qui possède le taux de marge le plus élevé.

8.4. Classement des produits

1. Classez les produits par chiffre d'affaires net décroissant.
2. Utilisez une fonction de fenêtrage Spark pour calculer le rang.
3. Comparez `rank`, `dense_rank` et `row_number`.
4. Proposez un classement par catégorie.
5. Expliquez le traitement des égalités de chiffre d'affaires.
6. Définissez le traitement des produits sans vente éligible.

Livrable : tables Gold, indicateurs globaux et par catégorie, classements et interprétation métier.

9. Étape 6 : Spark SQL

Enregistrez les DataFrames nécessaires sous forme de vues temporaires et réalisez les requêtes suivantes en Spark SQL :

1. Afficher les commandes éligibles et leur date.
2. Calculer le chiffre d'affaires net global.
3. Calculer le chiffre d'affaires, le coût et la marge par catégorie.
4. Identifier les produits les plus performants.
5. Calculer le chiffre d'affaires par mois.
6. Identifier les clients ayant généré le chiffre d'affaires le plus élevé.
7. Calculer le classement des produits par catégorie à l'aide d'une fonction de fenêtrage.
8. Comparer les résultats SQL aux résultats obtenus avec les DataFrames.

Questions

- Quels avantages Spark SQL apporte-t-il à l'analyse ?
- Pourquoi les requêtes SQL et DataFrames doivent-elles produire des résultats cohérents ?
- Quelles précautions prendre avec les montants décimaux ?
- Quelle différence existe entre une agrégation globale et une agrégation par groupe ?

Livrable : requêtes SQL, résultats observés et comparaison avec les calculs DataFrame.

10. Étape 7 : Réconciliation et validation

10.1. Réconciliation financière

Vérifiez que :

Marge totale = CA net total − Coût total.

Comparez les résultats obtenus :

- à partir de la vue détaillée des lignes ;
- à partir des agrégations DataFrame ;
- à partir des requêtes Spark SQL ;
- après relecture des tables Gold persistées.

Expliquez toute différence. Les comparaisons doivent utiliser des conventions d'arrondi cohérentes.

10.2. Résultats de référence

Les résultats de référence sur le jeu E-shop sont les suivants :

Indicateur	Résultat attendu
Commandes éligibles	20
Lignes de commande Gold	40
Clients concernés	20
Quantité vendue	120
Chiffre d'affaires net	13 259,80
Coût total	9 150,00
Marge	4 109,80
Taux de marge sur CA	30,99 %
Panier moyen	662,99

Ces résultats supposent que les règles métier du sujet sont appliquées, que les doublons exacts sont traités sans supprimer leur occurrence canonique, que les références invalides ne contribuent pas à Gold et que les promotions inconnues ou non valides dans le temps donnent lieu à une remise nulle.

En cas d'écart, recherchez la cause : filtres, jointures, doublons, dates de validité des promotions, conversions numériques, prix transactionnels ou formules financières.

Il est interdit de modifier artificiellement les résultats pour les faire correspondre aux valeurs de référence.

Livrable : rapport de réconciliation présentant les résultats obtenus, les valeurs de référence, les écarts et leurs explications.

11. Étape 8 : Persistance, reproductibilité et idempotence

Travail demandé

1. Écrivez les données Bronze, Silver, Gold et de quarantaine dans un stockage persistant.
2. Documentez les formats et les chemins utilisés.
3. Arrêtez l'application Spark et démarrez une nouvelle session.
4. Relisez les données persistées et vérifiez leurs volumes.
5. Réexécutez le pipeline sur les mêmes sources.
6. Vérifiez que la réexécution ne crée pas de doublons et ne modifie pas les résultats de manière inattendue.
7. Documentez la stratégie de remplacement ou de mise à jour des tables.
8. Vérifiez que le manifeste permet de retrouver les informations d'ingestion correspondantes.

Questions

- Quelle différence existe entre un DataFrame en mémoire et une table persistée ?
- Pourquoi le cache Spark ne constitue-t-il pas une preuve de persistance ?
- Qu'est-ce que l'idempotence d'un pipeline de données ?

- Quels problèmes peuvent survenir lors d'une écriture répétée dans un dossier non nettoyé ?
- Comment éviter de mélanger les données de deux exécutions ?

Livrable : preuves de persistance, résultats après redémarrage et démonstration de reproductibilité.

12. Rapport final et livrables

Le rendu doit contenir :

1. le notebook ou les scripts PySpark exécutables ;
2. la configuration de l'environnement et la procédure de lancement ;
3. les schémas utilisés et les règles de qualité ;
4. les tables Bronze, Silver, Gold et de quarantaine ;
5. le rapport de qualité et les volumes observés ;
6. le manifeste d'ingestion ;
7. les KPI globaux et par catégorie ;
8. les requêtes Spark SQL et les fonctions de fenêtrage ;
9. le rapport de réconciliation ;
10. les preuves de persistance et d'idempotence ;
11. une synthèse métier présentant les principaux résultats et les limites de l'analyse.

Chaque résultat doit être reproductible à partir des sources fournies et des traitements remis. Les captures d'écran seules ne constituent pas un rendu suffisant.

13. Organisation attendue du projet

```
E-shop/
├── data/
│   └── source/
│       ├── customers.csv
│       ├── products.csv
│       ├── orders.csv
│       ├── order_items.csv
│       └── promotions.jsonl
├── notebooks/
│   └── TP_E-shop.ipynb
├── scripts/
├── data_lakehouse/
            ├── bronze/
            ├── silver/
            ├── gold/
            ├── quarantine/
            └── metadata/
```

Vous pouvez adapter cette organisation, à condition de conserver une séparation claire entre sources, traitements, données persistées et livrables.

Annexe A : Présentation du jeu de données et anomalies intentionnelles

Les fichiers `customers.csv`, `products.csv`, `orders.csv`, `order_items.csv` et `promotions.jsonl` fournis avec le sujet constituent le jeu de données de référence.

Ils doivent être enregistrés dans le répertoire `E-shop/data/source/`, en conservant exactement les lignes, les noms de colonnes, les séparateurs et les formats fournis. Les anomalies sont intentionnelles : elles doivent être traitées dans le pipeline et non corrigées manuellement.

Les volumes annoncés pour le jeu de données de référence sont les suivants :

Source	Volume annoncé
`customers.csv`	32 lignes
`products.csv`	14 lignes
`orders.csv`	44 lignes
`order_items.csv`	86 lignes
`promotions.jsonl`	6 lignes

A.1. Anomalies intentionnelles

Clients (`customers.csv`) :

- doublon exact de `C005` ;
- nom vide pour `C031` ;
- date d'inscription invalide pour `C031`.

Produits (`products.csv`) :

- doublon exact de `P003` ;
- coût négatif pour `P013`.

Commandes (`orders.csv`) :

- doublon exact de `O0003` ;
- client inexistant pour `O9001` ;
- date invalide pour `O9002` ;
- statut non reconnu pour `O9003` ;
- codes promotionnels inconnus présents dans certaines commandes.

Les statuts reconnus sont `PAID`, `DELIVERED`, `CANCELLED` et `PENDING`. Tout autre statut doit être considéré comme non reconnu.

Lignes de commande (`order_items.csv`) :

- doublon exact de `I0004` ;

- commande inexistante pour `I9001` ;
- produit inexistant pour `I9002` ;
- quantité négative pour `I9003` ;
- quantité non numérique pour `I9004` ;
- quantité nulle pour `I9005`.

Promotions (`promotions.jsonl`) :

- les cinq codes du référentiel sont `PROMO5`, `PROMO10`, `SUMMER15`, `VIP20` et `OLD30` ;
- les périodes de validité diffèrent selon les promotions ;
- certaines commandes utilisent un code inconnu ou une promotion expirée.
- Le fichier contient une sixième ligne volontairement mal formée, qui doit être détectée et placée en quarantaine.

A.2. Rappel des fichiers sources

Les fichiers sources complets sont présentés dans l'annexe B. Ils doivent être utilisés tels quels, sans modification manuelle.

Vérifiez les volumes réellement lus par Spark au début du TP. Les volumes annoncés correspondent au nombre de lignes de données, hors ligne d'en-tête pour les fichiers CSV.

Consigne finale

Les données de E-shop sont volontairement imparfaites. Votre travail consiste à construire un pipeline de Data Engineering fiable, à expliciter les contrôles, à justifier les décisions de nettoyage et à produire des indicateurs traçables.

La qualité du rendu sera évaluée autant sur la justesse des résultats que sur la rigueur des contrôles, la reproductibilité des traitements et la capacité à expliquer les décisions prises.

Annexe B : Contenu intégral des fichiers sources

Les fichiers ci-dessous constituent les sources à placer dans le répertoire `E-shop/data/source/`. Les anomalies sont intentionnelles et doivent être traitées dans le pipeline, et non corrigées manuellement.

B.1. `customers.csv`

```
customer_id;customer_name;region;registration_date
C001;Alice 01;Nord;2024-01-08
C002;Youssef 02;Sud;2024-01-15
C003;Sara 03;Centre;2024-01-22
C004;Omar 04;Est;2024-01-29
C005;Nadia 05;Ouest;2024-02-05
C006;Amine 06;Nord;2024-02-12
C007;Alice 07;Sud;2024-02-19
```

```
C008;Youssef 08;Centre;2024-02-26
C009;Sara 09;Est;2024-03-04
C010;Omar 10;Ouest;2024-03-11
C011;Nadia 11;Nord;2024-03-18
C012;Amine 12;Sud;2024-03-25
C013;Alice 13;Centre;2024-04-01
C014;Youssef 14;Est;2024-04-08
C015;Sara 15;Ouest;2024-04-15
C016;Omar 16;Nord;2024-04-22
C017;Nadia 17;Sud;2024-04-29
C018;Amine 18;Centre;2024-05-06
C019;Alice 19;Est;2024-05-13
C020;Youssef 20;Ouest;2024-05-20
C021;Sara 21;Nord;2024-05-27
C022;Omar 22;Sud;2024-06-03
C023;Nadia 23;Centre;2024-06-10
C024;Amine 24;Est;2024-06-17
C025;Alice 25;Ouest;2024-06-24
C026;Youssef 26;Nord;2024-07-01
C027;Sara 27;Sud;2024-07-08
C028;Omar 28;Centre;2024-07-15
C029;Nadia 29;Est;2024-07-22
C030;Amine 30;Ouest;2024-07-29
C005;Nadia 05;Ouest;2024-02-05
C031;;Nord;2025-02-30
```

Anomalies à détecter : doublon de `C005`, nom vide pour `C031` et date invalide pour `C031`.

B.2. `products.csv`

```
product_id;product_name;category;unit_price;unit_cost
P001;Laptop;Informatique;800.00;600.00
P002;Mouse;Accessoires;20.00;8.00
P003;Screen;Informatique;200.00;140.00
P004;Keyboard;Accessoires;45.00;25.00
P005;Tablet;Informatique;320.00;230.00
P006;Headphones;Audio;75.00;40.00
P007;Webcam;Accessoires;55.00;30.00
P008;Printer;Bureau;180.00;125.00
P009;SSD 1TB;Stockage;95.00;58.00
P010;Router;Réseau;110.00;70.00
P011;Microphone;Audio;65.00;35.00
P012;USB Hub;Accessoires;25.00;11.00
P003;Screen;Informatique;200.00;140.00
P013;Broken Cost Item;Divers;30.00;-4.00
```

Anomalies à détecter : doublon de `P003` et coût négatif pour `P013`.

B.3. `orders.csv`

```
order_id;customer_id;order_date;status;promotion_code
O0001;C007;2025-05-04;PAID;
O0002;C014;2025-05-07;DELIVERED;PROMO5
O0003;C021;2025-05-10;CANCELLED;PROMO10
O0004;C028;2025-05-13;PENDING;SUMMER15
O0005;C005;2025-05-16;PAID;VIP20
O0006;C012;2025-05-19;DELIVERED;UNKNOWN
O0007;C019;2025-05-22;CANCELLED;OLD30
```

```
O0008;C026;2025-05-25;PENDING;
O0009;C003;2025-05-28;PAID;PROMO5
O0010;C010;2025-05-31;DELIVERED;PROMO10
O0011;C017;2025-06-03;CANCELLED;SUMMER15
O0012;C024;2025-06-06;PENDING;VIP20
O0013;C001;2025-06-09;PAID;UNKNOWN
O0014;C008;2025-06-12;DELIVERED;OLD30
O0015;C015;2025-06-15;CANCELLED;
O0016;C022;2025-06-18;PENDING;PROMO5
O0017;C029;2025-06-21;PAID;PROMO10
O0018;C006;2025-06-24;DELIVERED;SUMMER15
O0019;C013;2025-06-27;CANCELLED;VIP20
O0020;C020;2025-06-30;PENDING;UNKNOWN
O0021;C027;2025-07-03;PAID;OLD30
O0022;C004;2025-07-06;DELIVERED;
O0023;C011;2025-07-09;CANCELLED;PROMO5
O0024;C018;2025-07-12;PENDING;PROMO10
O0025;C025;2025-07-15;PAID;SUMMER15
O0026;C002;2025-07-18;DELIVERED;VIP20
O0027;C009;2025-07-21;CANCELLED;UNKNOWN
O0028;C016;2025-07-24;PENDING;OLD30
O0029;C023;2025-07-27;PAID;
O0030;C030;2025-07-30;DELIVERED;PROMO5
O0031;C007;2025-08-02;CANCELLED;PROMO10
O0032;C014;2025-08-05;PENDING;SUMMER15
O0033;C021;2025-08-08;PAID;VIP20
O0034;C028;2025-05-03;DELIVERED;UNKNOWN
O0035;C005;2025-05-06;CANCELLED;OLD30
O0036;C012;2025-05-09;PENDING;
O0037;C019;2025-05-12;PAID;PROMO5
O0038;C026;2025-05-15;DELIVERED;PROMO10
O0039;C003;2025-05-18;CANCELLED;SUMMER15
O0040;C010;2025-05-21;PENDING;VIP20
O0003;C021;2025-05-10;CANCELLED;PROMO10
O9001;C999;2025-06-20;PAID;
O9002;C001;2025-02-30;PAID;
O9003;C002;2025-06-20;IN_PROGRESS;
```

Anomalies à détecter : doublon de `O0003`, client inexistant pour `O9001`, date invalide pour `O9002` et statut non reconnu pour `O9003`.

B.4. `order_items.csv`

```
order_item_id;order_id;product_id;quantity;unit_price
I0001;O0001;P003;2;200.00
I0002;O0001;P008;3;180.00
I0003;O0002;P006;3;75.00
I0004;O0002;P011;4;65.00
I0005;O0003;P009;4;95.00
I0006;O0003;P002;1;20.00
I0007;O0004;P012;1;25.00
I0008;O0004;P005;2;320.00
I0009;O0005;P003;2;190.00
I0010;O0005;P008;3;180.00
I0011;O0006;P006;3;75.00
I0012;O0006;P011;4;65.00
I0013;O0007;P009;4;95.00
I0014;O0007;P002;1;20.00
I0015;O0008;P012;1;25.00
```

```
I0016;O0008;P005;2;320.00
I0017;O0009;P003;2;200.00
I0018;O0009;P008;3;171.00
I0019;O0010;P006;3;75.00
I0020;O0010;P011;4;65.00
I0021;O0011;P009;4;95.00
I0022;O0011;P002;1;20.00
I0023;O0012;P012;1;25.00
I0024;O0012;P005;2;320.00
I0025;O0013;P003;2;200.00
I0026;O0013;P008;3;180.00
I0027;O0014;P006;3;71.25
I0028;O0014;P011;4;65.00
I0029;O0015;P009;4;95.00
I0030;O0015;P002;1;20.00
I0031;O0016;P012;1;25.00
I0032;O0016;P005;2;320.00
I0033;O0017;P003;2;200.00
I0034;O0017;P008;3;180.00
I0035;O0018;P006;3;75.00
I0036;O0018;P011;4;61.75
I0037;O0019;P009;4;95.00
I0038;O0019;P002;1;20.00
I0039;O0020;P012;1;25.00
I0040;O0020;P005;2;320.00
I0041;O0021;P003;2;200.00
I0042;O0021;P008;3;180.00
I0043;O0022;P006;3;75.00
I0044;O0022;P011;4;65.00
I0045;O0023;P009;4;90.25
I0046;O0023;P002;1;20.00
I0047;O0024;P012;1;25.00
I0048;O0024;P005;2;320.00
I0049;O0025;P003;2;200.00
I0050;O0025;P008;3;180.00
I0051;O0026;P006;3;75.00
I0052;O0026;P011;4;65.00
I0053;O0027;P009;4;95.00
I0054;O0027;P002;1;19.00
I0055;O0028;P012;1;25.00
I0056;O0028;P005;2;320.00
I0057;O0029;P003;2;200.00
I0058;O0029;P008;3;180.00
I0059;O0030;P006;3;75.00
I0060;O0030;P011;4;65.00
I0061;O0031;P009;4;95.00
I0062;O0031;P002;1;20.00
I0063;O0032;P012;1;23.75
I0064;O0032;P005;2;320.00
I0065;O0033;P003;2;200.00
I0066;O0033;P008;3;180.00
I0067;O0034;P006;3;75.00
I0068;O0034;P011;4;65.00
I0069;O0035;P009;4;95.00
I0070;O0035;P002;1;20.00
I0071;O0036;P012;1;25.00
I0072;O0036;P005;2;304.00
I0073;O0037;P003;2;200.00
I0074;O0037;P008;3;180.00
I0075;O0038;P006;3;75.00
I0076;O0038;P011;4;65.00
```

```
I0077;O0039;P009;4;95.00
I0078;O0039;P002;1;20.00
I0079;O0040;P012;1;25.00
I0080;O0040;P005;2;320.00
I0004;O0002;P011;4;65.00
I9001;O9999;P001;1;800.00
I9002;O0001;P999;2;10.00
I9003;O0002;P002;-2;20.00
I9004;O0004;P004;abc;45.00
I9005;O0005;P005;0;320.00
```

Anomalies à détecter : doublon de `I0004`, commande inexistante pour `I9001`, produit inexistant pour `I9002`, quantité négative pour `I9003`, quantité non numérique pour `I9004` et quantité nulle pour `I9005`.

B.5. `promotions.jsonl`

Enregistrez chaque objet ci-dessous sur une ligne distincte dans le fichier `promotions.jsonl`.

```
{"promotion_code":"PROMO5","discount_rate":0.05,"start_date":"2025-01-
01","end_date":"2025-12-31"}
{"promotion_code":"PROMO10","discount_rate":0.1,"start_date":"2025-05-
01","end_date":"2025-05-31"}
{"promotion_code":"SUMMER15","discount_rate":0.15,"start_date":"2025-06-
01","end_date":"2025-08-31"}
{"promotion_code":"VIP20","discount_rate":0.2,"start_date":"2025-01-
01","end_date":"2025-12-31"}
{"promotion_code":"OLD30","discount_rate":0.3,"start_date":"2024-01-
01","end_date":"2024-12-31"}
{"promotion_code":"BAD_JSON","discount_rate":0.10,"start_date":"2025-06-
01","end_date":
```

Les promotions `PROMO5`, `PROMO10`, `SUMMER15`, `VIP20` et `OLD30` constituent les codes présents dans le référentiel. Certaines commandes contiennent toutefois le code `UNKNOWN`, qui doit être traité conformément aux règles de qualité et de gestion des promotions définies dans le sujet.

B.6. Organisation des fichiers

L'organisation proposée pour le projet est la suivante :

```
E-shop/
├── data/
│   └── source/
│       ├── customers.csv
│       ├── products.csv
│       ├── orders.csv
│       ├── order_items.csv
│       └── promotions.jsonl
├── notebooks/
│   └── TP_E-shop.ipynb
├── scripts/
└── data_lakehouse/
    ├── bronze/
    ├── silver/
    ├── gold/
```

```
├── quarantine/
└── metadata/
```

Le répertoire `data/source/` contient les fichiers d'origine. Les répertoires `bronze/`, `silver/` et `gold/` accueillent respectivement les données brutes ingérées, les données nettoyées et structurées, puis les données préparées pour l'analyse. Le répertoire `quarantine/` conserve les enregistrements rejetés ou nécessitant un traitement particulier, tandis que `metadata/` contient les informations de suivi et de traçabilité des traitements.